

•



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Adding rules

On this page

Adding rules

Was this page helpful? 

Why

Rules add an extra layer of scoring and decision-making to your project in runtime. Just like [Models](#), **Rules** contribute to the classification of events moving along the **Workflow** and to the final [Workflow outcomes](#).

Once you reach a [Rules Evaluation](#) that shows that your **Rules** block has reached or surpassed business objectives, you can simply configure the same block of Rules to run within a service in runtime.

You can deploy blocks of [Rules](#) in a runtime **Workflow** by adding **Rules Services**. Typically, you can deploy different **Rules Services** both for detection, for example, blacklisting users, and for a decision, for example weighing in on the final score.

How to

To add and configure a block of [Rules](#) to score events in your **Workflow**, execute the following actions:

1. In the **Workflows** tab, select an existing **Workflow**.
2. Drag a new **Rules Service** to the **Diagram** canvas.



Rules Services

When selecting Rules Services in workflows, a snapshot is automatically generated and selected in the configuration dialog. You cannot select any other snapshot than the automatically-generated one.

3.  **Edit the Rules Service.**

4. In the **Main** tab:

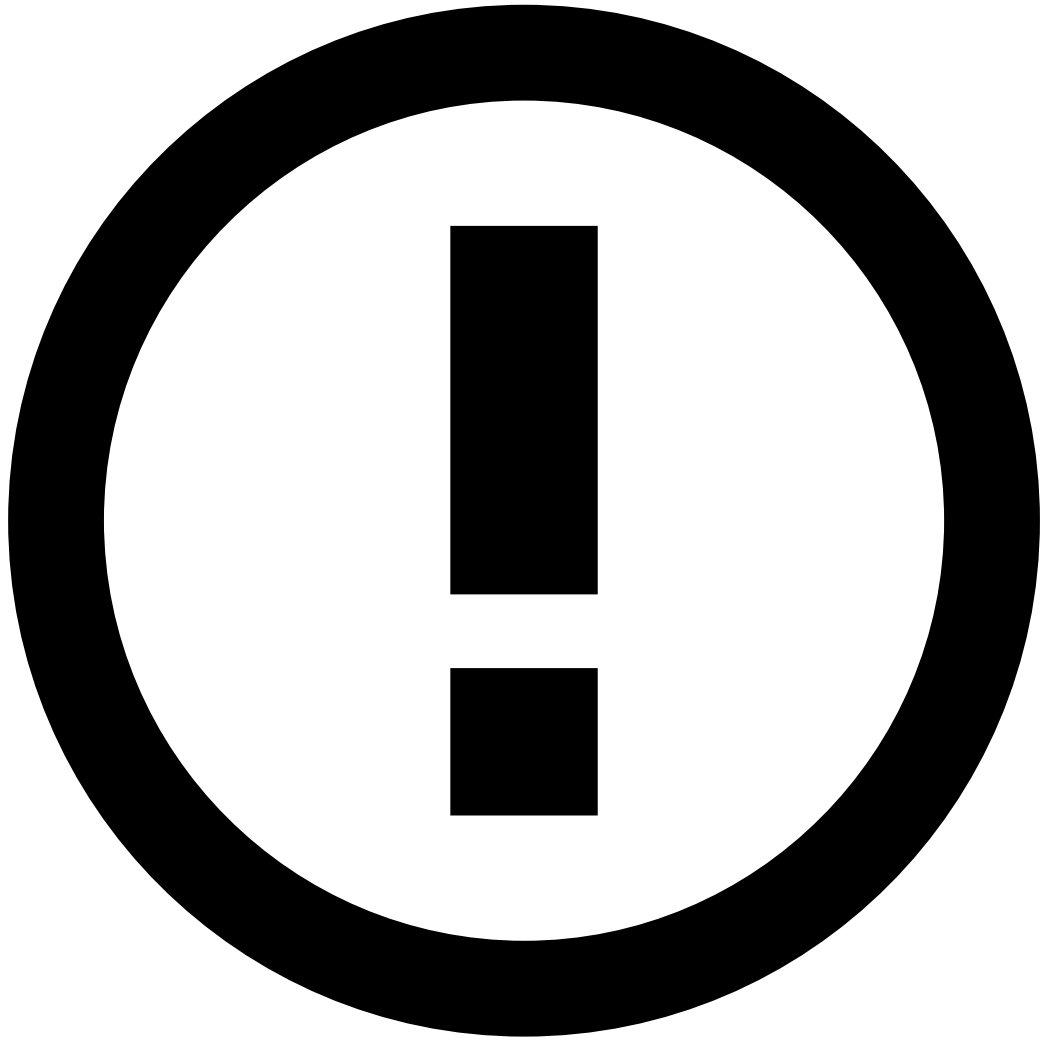
1. Define to which of the [Workflow Outcomes](#) the **Rules** will contribute. After being processed by a **Rules Service**, the `<outcome_name>` and `<outcome_name>_score` fields in the event schema will reflect the **Rules** decision.
2. Select the **Rules Project** that contains the block of **Rules** you want to apply in your runtime processing. The **Rules Project** also defines the input schema for the **Rules Service** as defined at the Data Science stage.



Self-Service Rules

All usage of self-services interface is performed through a Rules Workflow Element.

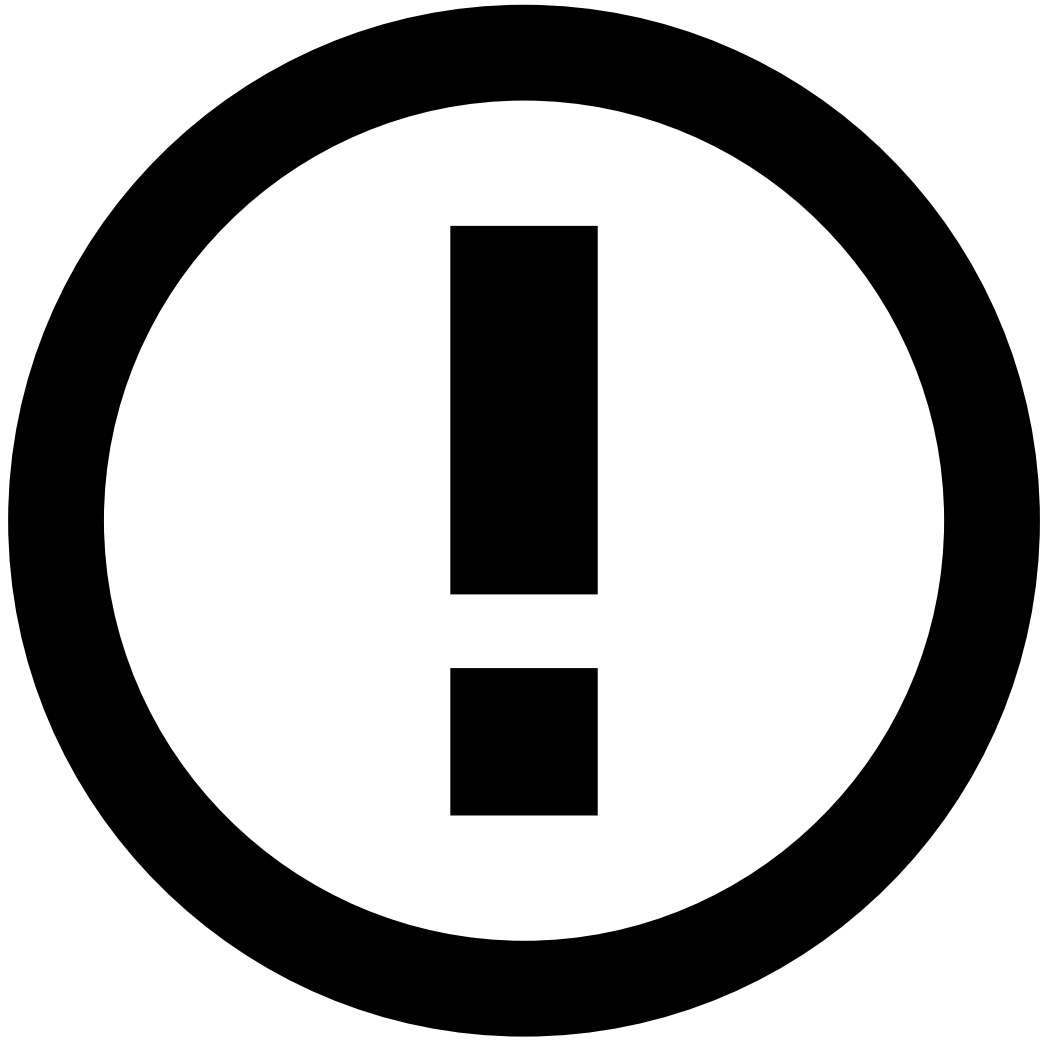
When adding a [Self-Service Rules project](#), a snapshot is automatically created and assigned. The latest snapshot is built adding changes on top of the one published before it (as reflected in the Publishing section). Older snapshots are kept and can be changed via API to allow a more sophisticated and greater control over the system. This allows you to easily look up the history of the published rules and roll back, if needed.



Self-Service Rules in a Multi-Tenancy scenario

In order to have a workflow ready for Self-Service Rules in a Multi-Tenanted scenario, you should add two rule elements:

- One pointing to a Global Simple rules projects
 - One pointing to a By-tenant Simple rules project.
3. Select the **Rules Snapshot** to specify which **Rules** to execute in the **Rules Service**. The **Rules Snapshot** carries the **Rules**, their order of execution and individual contribution to outcomes and scores, as defined and evaluated during the Data Science stage.



Multi-Tenancy Solution

If the Workflow is being configured with a [Rules Project](#) of the type Multi-Tenancy, the **Rules Snapshot** has information related to the **Rules** that are going to be applied to each Tenant.

4. Configure whether **Workflow** generates **Explanations** as to why the **Rules** triggered.



Explanations on critical path

If you choose to compute **Explanations** on the **Critical Path**, they will immediately be accessible to be processed in **Custom Service**, once they are generated.

5. In the **Schemas** tab:

1. Confirm that the **Service Input** and **Service Output** are consistent with the data you expect for the **Rules**.
2. Make sure the **Service Input** and **Service Output** are correctly mapped from the **Event Schema**, for each real-time event moving along the **Workflow**.

6. In the **Properties** tab, set the values for each property to define the service configuration in your **Workflow**. Properties and default values are defined by Pulse.

7.  Connect the previous service in the Workflow to the Rules Service. You can also learn [to organize your work with blocks](#) in order to do this in a most efficient way.

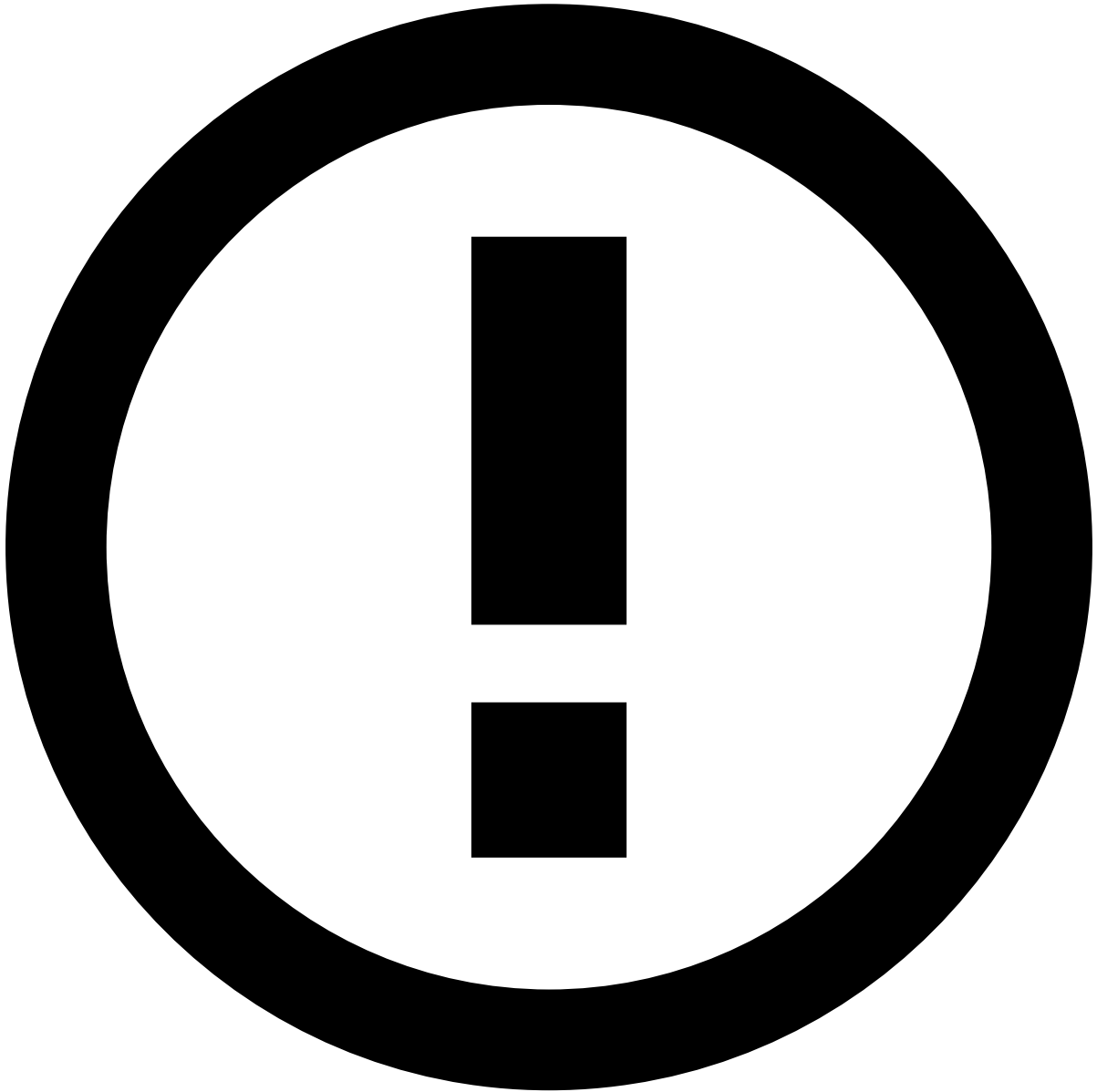
Solving schema conflicts

If Pulse invalidates your [Workflow](#) due to schema mapping conflicts in a Workflow Service, you must solve them as follows:

1.  Edit the **Service**.
2. Select the **Schemas** tab.

3. Look for a count of **Input Conflicts** and **Output Conflicts** above 0.
4. Select each **Conflict** filter to see which fields could not be mapped. Alternatively, select the **Service** filter and locate the conflict fields in the **Event Schema**. These are marked as *Missing Input* or *Missing Output*.
5. If the **Event Schema** contains a field that you can map to the **Service Input** or **Output**, manually enter the name of that field.

The following images display schema conflicts during resolution:



Conflicts count

Note that Pulse will still display the original conflicts count even after you have solved them.



Removed fields

If a **Plan** you configured for a **Plan Service** removes fields from the workflow schema, then you must ensure that subsequent services in the **Workflow** do not require those fields. Otherwise, Pulse will generate schema conflicts, invalidate your **Workflow**, and you will have no available fields to map from.

In these cases, you should redesign your **Plan**, articulate it with the remaining Data Science work to be used in runtime, and redeploy it to your **Workflow**.

Next Steps

Proceed with the designing of your Workflow by adding, configuring and connecting other services. Check the following pages:

- [Adding input adapters](#)
- [Adding plans](#)
- [Adding models](#)
- [Adding custom services](#)